

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

«Санкт-Петербургский политехнический университет Петра Великого»

«Высшая школа программной инженерии»

ОТЧЕТ

о прохождении учебной практики

«Разработка генератора конфигурационных файлов для сервиса нагрузочного
тестирования Yandex.Tank»

Выполнил:

студент

группы 5130904/40005

Костюков М.Е.

Руководитель практики:

Касилов В.А.

Содержание

Введение	3
Выбранный технологический стек	4
Описание работы программы	5
Возникшие сложности	6
Выводы по работе	7
Список используемых источников	8
Приложение	9

Введение

При разработке и эксплуатации современных веб-сервисов одним из важных этапов нагрузочное тестирование. Оно позволяет оценить стабильность работы сервиса и его максимальную нагрузку. Популярным инструментом для решения таких задач является Yandex.Tank.

Тем не менее, написание конфигураций для Yandex.Tank представляет собой монотонную и часто трудоемкую работу (особенно для больших сервисов).

Разрабатываемое приложение направлено на автоматизацию этого процесса. Система принимает на вход готовую спецификацию веб-сервиса в формате OpenAPI (ранее Swagger), параметры планируемой нагрузки и ограничения, после чего автоматически генерирует готовый к запуску архив с конфигурационными файлами и окружением для запуска. Это позволяет ускорить подготовку к нагрузочным тестам и уменьшить трудозатраты тестировщика.

Выбранный технологический стек

Для написания приложения использовались следующие технологии:

Компонент	Назначение компонента	Версия
Java	Язык разработки	25
Spring	Веб-фреймворк	4.1.0
Thymeleaf	Шаблонизатор для отображения пользовательского интерфейса	3.1.5
Bootstrap	Библиотека для верстки веб-интерфейса	5.3.3
Swagger Parser V3	Чтение и парсинг OpenAPI спецификаций	2.1.45
DataFaker	Генерация реалистичных тестовых данных	2.7.0
Docker	Контейнеризация приложения	29.5.3
Yandex.Tank	Основной инструмент проведения нагрузочных тестов	latest
InfluxDB	База данных для сбора отчетов о тесте	1.8
Grafana	Платформа для визуализации результатов тестирования	8.5.27

Описание работы программы

1. **Прием исходных данных.** Пользователь взаимодействует с приложением через веб-интерфейс или REST API. В форме (или в теле запроса) передается файл спецификации OpenAPI (в формате JSON или YAML) и набор параметров тестирования: адрес тестируемого сервиса, схема и профиль нагрузки, параметры автоматической остановки теста.
2. **Парсинг OpenAPI.** Программа с помощью библиотеки SwaggerParser для каждого обнаруженного пути и связанного с ним HTTP-метода составляет модель запроса к Yandex.Tank.
3. **Генерация тестовых данных.** Чтобы сделать тест реалистичным используется библиотека DataFaker. Она, в зависимости от типа и имени запрашиваемого объекта, создает уникальные значения, такие как: имена, номера телефонов, адреса почты, id и пр. На каждый эндпоинт по умолчанию создается несколько (25) вариантов запросов, что увеличивает их вариативность.
4. **Формирование конфигов Yandex.Tank.** На основе сгенерированных запросов программа собирает `ammo.txt`, а с помощью введенных пользователем данных собирает конфигурацию `load.yaml`.
5. **Сборка архива.** Завершив предыдущие шаги сервис упаковывает сформированные файлы `ammo.txt` и `load.yaml`, а также `README.md`, и файлы быстрого запуска в zip-архив и отдает его пользователю в виде скачиваемого файла.

Полный цикл работы с точки зрения пользователя включает запуск своего тестируемого сервиса, запуск этого приложения для генерации конфигов, и последующее изучение полученных графиков в Grafana.

Возникшие сложности

В процессе разработки приложения возник ряд трудностей:

1. **Парсинг OpenAPI-документации.** Спецификации OpenAPI могут иметь большую степень вложенности. Использование библиотеки `swagger-parser` упрощает чтение файла, но построение тела запроса для тестирования требует обхода дерева схемы. Для решения был написан рекурсивный метод, который при обнаружении типов `object` или `array` вызывал сам себя, постепенно спускаясь по иерархии типов.
2. **Реалистичные данные.** Если отправлять сервису во время теста однотипные запросы с заглушками, тест будет иметь мало общего с реальной нагрузкой. Поэтому для заполнения JSON-структур реальными данными была использована библиотека `DataFaker`, а также реализована проверка имен полей: если имя содержит подстроки «`id`», «`age`», «`price`», «`email`», «`phone`» или «`address`», алгоритм подставляет случайные значения соответствующего типа.
3. **Недоступность Overload.** Официальная документация Yandex.Tank предлагает отправлять результаты тестирования во внешнее хранилище Overload. Однако, на момент разработки, сервис оказался недоступен из-за чего проанализировать результаты теста пользователю было бы проблематично. Решение включает в себя добавление в `zip`-архив файла для Docker Compose, поднимающего **InfluxDB** и **Grafana**. В состав архива входит готовый простой дашборд, который визуализирует квантили времени ответа, количество успешных и неуспешных HTTP/сетевых кодов ответов, а также активные потоки.
4. **Идеи для дальнейшего развития.** Обработка рекурсивных моделей: OpenAPI-спецификация может содержать бесконечную рекурсию (например, модель может содержать массив объектов того же типа, что и сама модель) текущий алгоритм не сможет это обработать.

Выводы по работе

В ходе выполнения работы было разработано веб-приложение для генерации конфигурационных файлов инструмента нагрузочного тестирования Yandex.Tank на основе спецификаций OpenAPI.

В процессе работы над проектом были приобретены следующие практические навыки:

1. Получен опыт работы со Spring MVC.
2. Работа со спецификациями OpenAPI. Изучена внутренняя структура документов Swagger и освоены методы программного разбора данных спецификаций.
3. Конфигурирование нагрузочного тестирования для Yandex.Tank. Изучен синтаксис конфигурационных файлов и формат атмо-файлов.

Разработанный инструмент представляет собой готовое базовое решение, способное сократить время подготовки однотипных тестов.

Список используемых источников

1. Документация по инструменту нагрузочного тестирования Yandex.Tank:
URL: <https://yandextank.readthedocs.io/en/latest/tutorial.html>
2. Пример нагрузочного тестирования сайта:
URL: <https://serveradmin.ru/primer-nagruzochnogo-testirovaniya-sajta-s-yandex-tank/>
3. Разбор спецификации OpenAPI:
URL: <https://habr.com/ru/companies/first/articles/973186/>
4. Введение в фреймворк Spring Boot:
Книга «Spring быстро» Лауренциу Спилкэ

Приложение

The screenshot shows the 'YT-config-generator' application in a web browser. The title is 'YT-config-generator' with a subtitle: 'Заполните параметры теста, загрузите файл спецификации OpenAPI и скачайте готовый архив настроек для Yandex.Tank.' The interface is divided into two main sections:

- 1. Спецификация (Specification):** Contains fields for 'Файл OpenAPI' (File OpenAPI) with a value 'api-docs.json', 'Адрес тестируемого сервиса' (Service address) with 'host.docker.internal:8080', and 'Токен аутентификации' (Authentication token) with 'eyJhbGciOiJIUzI1NiIsInR5cCI6Ikpz...'. A note mentions adding an 'Authorization: Bearer' header.
- 2. Профиль нагрузки (Load Profile):** Contains fields for 'Длительность теста' (Test duration) set to 30, 'Тип нагрузки' (Load type) set to 'RPS (запросы в секунду)', 'Схема нагрузки' (Load schema) set to 'Константная нагрузка', 'Запросы/сек (RPS)' set to 50, 'RPS на старте' set to 5, 'RPS в конце' set to 60, 'Шаг RPS' set to 10, and 'Количество потоков' (Number of threads) set to 1000. Each field has a small icon for configuration.

Рис. 1. Пользовательский интерфейс приложения

The screenshot shows the '3. Авто-остановка теста (Autostop)' section of the application. It features a toggle switch 'Активировать автоматическую остановку теста' (Activate automatic test stop) which is turned on. Below this are three criteria for stopping the test:

- 1. Критерии по HTTP-кодам (Criteria by HTTP codes):** Fields for 'HTTP-код ошибки' (Error HTTP code) set to '5xx', 'Лимит ошибок' (Error limit) set to '25%', and 'Временное окно' (Time window) set to '10s'.
- 2. Критерии по Сетевым ошибкам (Criteria by Network errors):** Fields for 'Сетевой код ошибки' (Network error code) set to 'xx', 'Лимит ошибок' (Error limit) set to '25', and 'Временное окно' (Time window) set to '10s'.
- 3. Критерии по времени ответа (Latency) (Criteria by response time):** Fields for 'Максимальное время ответа' (Maximum response time) set to '1500ms' and 'Временное окно' (Time window) set to '10s'.

At the bottom, there is a large blue button labeled 'Сгенерировать и Скачать архив (ZIP)' (Generate and Download archive (ZIP)).

Рис. 2. Пользовательский интерфейс приложения

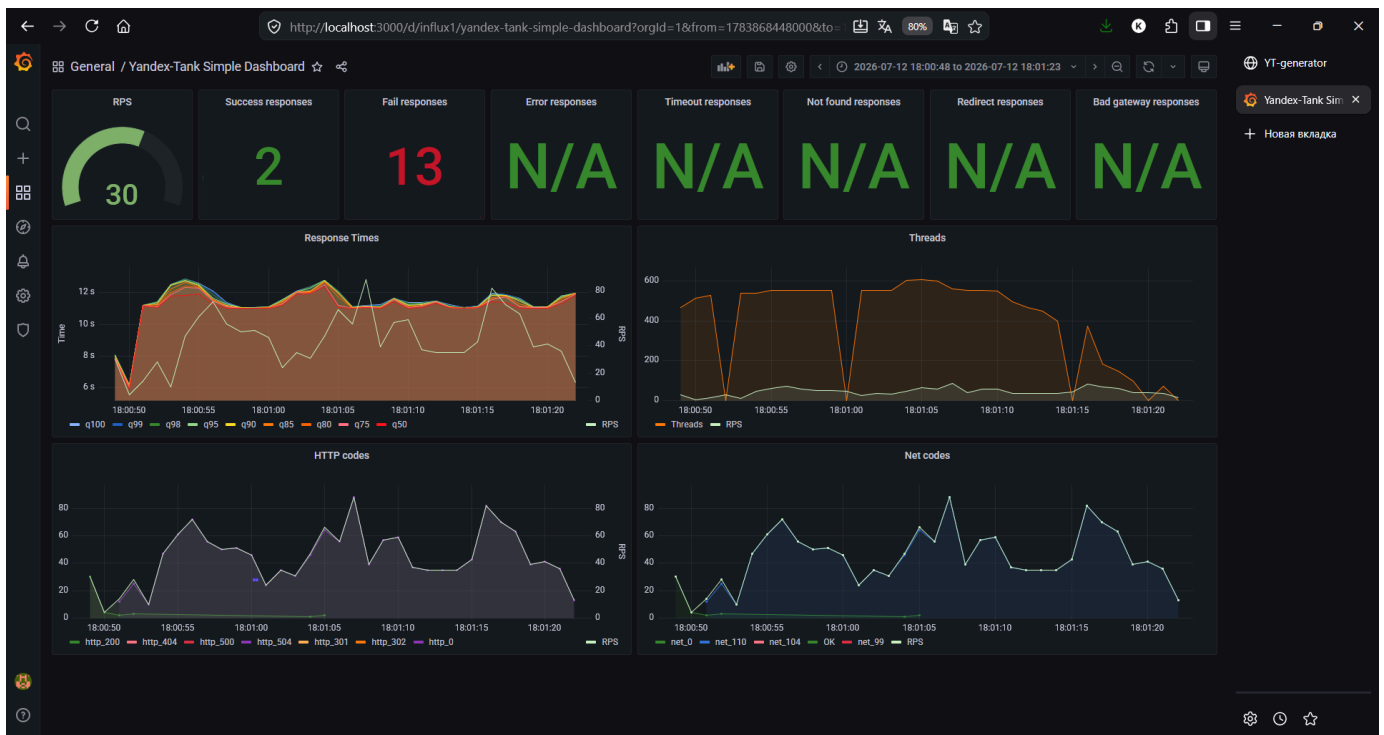


Рис. 3. Просмотр графиков в Grafana